

Fig. 6: Cannot distinguish the top one and the bottom one using the vector information between origin member and the reference member. Both top and bottom have the same angle and same distance along-axis, but they have different d2 and d4. (a): flipping along x-axis case. (b): flipping along y-axis case.

Algorithm 1 EDDR PM (Pattern Match Using Edge Driven Dissected Rectangle)

```

1: procedure EDDR-PM( $P_1, P_2, \dots, P_n$ , nonMem, PDB)
2:    $P_1$  = set of origin members in a layout
3:    $P_2$  = set of the first reference members in a layout
4:    $P_{3..n}$  = set of all the other reference members in a layout
5:   nonMem = set of non-member polygons in a layout
6:   PDB = a pattern description database.
7:   while !empty in  $P_1$  do
8:     Find a reference member  $p_2$  in  $P_2$  by searching the vector distance
9:     between  $P_1$ 's center and  $P_2$ 's center.(PDB has this info.)
10:    if found then
11:      if the found  $p_2$  a valid reference member then
12:        Create a bounding box using  $d_1, d_2, d_3$ , and  $d_4$  determined by
13:        vector info between  $P_1$  and the found  $P_2$ 's center.
14:      else
15:        No match. continue to next  $p_1$  in  $P_1$ 
16:      if nonMem exists inside the bounding box then
17:        No match. continue to next  $p_1$  in  $P_1$ 
18:      Find other members in  $P_3 \dots P_n$  inside the bounding box.
19:      for each member inside the bounding box do
20:         $n$  = number of each member inside bounding box
21:         $m$  = number of each member described in PDB
22:        if  $n \neq m$  then
23:          No match. continue to next  $p_1$  in  $P_1$ 
24:        if valid member == false then
25:          No match. continue to next  $p_1$  in  $P_1$ 
26:        Matching pattern found at this point.
27:        Output the bounding box to indicate the match.
28:        continue to next  $p_1$  in  $P_1$ 
29:      else
30:        No match. continue to next  $p_1$  in  $P_1$ 

```

If there is a non-member polygon inside the bounding box, it is immediately classified as no match (Fig. 7). To do this, we pass non-member polygons to EDDR_PM (Edge Driven Dissected Rectangle Pattern Match), which are created by OR and NOT operations. For example, non-members of Fig. 3 are:

$$\text{Non-member} = \text{Metal}_1 \text{ NOT }^1 (\text{OR } P_1 P_2 P_3 P_4 P_5 P_6)$$

At step (8) and (18) in the Algorithm 1, it uses scan line based topological check which is not ideal in terms of run time performance. We can improve it significantly by using a bin-search grid algorithm.

Another criterion we examine for an early invalidation of

¹Refer to Appendix A for NOT and OR operation

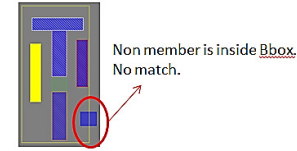


Fig. 7: When non-member interacts with the bounding box, it is immediately classified as no match.

Fig. 8: Immediate mismatch when the total number of each member inside the bounding box does not match.

a match for EDDR_PM is passing the total number of each member to EDDR_PM. If the number does not match inside the bounding box, it is classified as not a match right away (Fig. 8). We use information in PDB format as explained in Section III A for valid member check and creating the bounding box. At step (12) in the algorithm, we can determine which orientation we want among the 8 possible ones by calculating the vector (angle and distance) between P_1 and the found P_2 and referencing the information in PDB. If its plane is at along-axis or its angle is 45 degree, we create two bounding boxes and do the subsequent checks for each bounding box in the algorithm.

Because Algorithm 1 is a brute force search using the topological scan-lines, it is best to have as small number of P_1 as possible to reduce run time. We can do this by adding an extra edge operation related to other members when creating origin members. For example, we can add a space check between the origin member and some of the other reference members for final derivation of the origin members. This additional edge operation to reduce the total number of origin members in a layout does not incur much additional run time. However, it reduces EDDR_PM run time significantly.

Even though Algorithm 1 can do a decent job by reducing the number of P_1 , it is not sufficient to handle a huge number of P_1 rectangles presented in the layout. So, we developed another algorithm, Algorithm 2, which utilizes a bin-search grid technique. As presented in section IV, it achieved significant run time reduction.

C. Bin-Search Grid

A bin-search grid is efficient for finding objects which interact in a 2D space. It is typically much faster than topological scan-lines which must process all the objects in a single scan. It uses an adaptive structure for rectangle search via binning. The structure starts with a fixed pixel size but will re-grid more finely when the average number of objects in a bin becomes

Algorithm 2 EDDR PM (Pattern Match Using Edge Driven Dissected Rectangle)

```

1: procedure EDDR-PM(P1, P2, ..., Pn, nonMem, PDB)
2:   Inputs (P1..n, nonMem, and PDB) are the same as Algorithm 1.
3:   ADD_BIN for each member from P1 to Pn.
4:   LOCATE_BIN for all the origin members of P1 and get bin_counts
5:   for  $i = 1 \rightarrow \text{bin\_counts}$  for  $p_i$  in P1 bins do
6:     LOCATE_BIN a reference member, p2, in P2 bins by searching
7:     the vector distance between P1's center and P2's center. (PDB has this info.)
8:     if found then
9:       Same process as Algorithm 1 to decide match or no match
10:    .....
11:    LOCATE_BIN for other members inside the bounding box.
12:    .....
13:    Same process as Algorithm 1 to decide match or no match
14:    .....
15:   else
16:     No match. continue

```

excessive. Usually rectangular extents of geometric objects are used to insert into the grid. The grid is first populated with a series of ADD_BIN methods. ADD_BIN has a rectangle input along with an associated ID for the object. The grid can then be searched with the LOCATE_BIN method. LOCATE_BIN has a search rectangle as input, and returns a set of object IDs that interact with the rectangle. The bin-search grid has a fixed overall rectangular bounding box, usually layout extent, which is supplied by the client at initialization time. This rectangle is divided into a 2D grid with an initial default pixel size. ADD_BIN and LOCATE_BIN can then directly calculate the row and column elements of the grid to analyze using the pixel size. ADD_BIN contains heuristics to decrease the pixel size and recalculate the grid when the number of bins in the grid elements becomes large.

LOCATE_BIN analyzes the intersecting grid elements with a search rectangle and builds a list of unique bin IDs that have been previously added. The bin-search grid is efficient when the added bin extents, which are member rectangles in our case, are small relative to the overall layout extent bounding box. Since pattern bounding box is, in general, so small relative to the layout extent that it lies in one grid element in our application. Fig. 9 explains it graphically. In this picture, bounding box has 3 by 3 grid elements to locate bins inside it. Because LOCATE_BIN has $O(k)$ where k is the total number of grid elements overlapped by a search extent rectangle, it requires $O(9)$ to locate ID1 and ID2 bin.

D. Computational Complexity

Let n denote total number of P1 and let m denote total number of all members in a layout. Since Algorithm 1 visits all origin members in P1, it is $O(n)$ for the while loop. The scan-line search to find other members at the step (8) and step (18) inside the while loop requires m times topological check per each loop. Therefore, the complexity becomes $O(nm)$. Because of $m \geq n$, we can say that $O(n(n+c))$ where c is a constant, and it becomes $O(n^2)$.

Those two steps in Algorithm 1 have been replaced with LOCATE_BIN for Algorithm 2. Since LOCATE_BIN's time

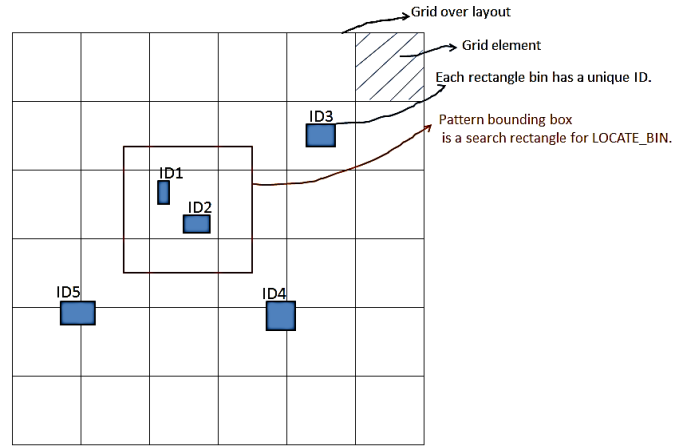


Fig. 9: Bin-search grid

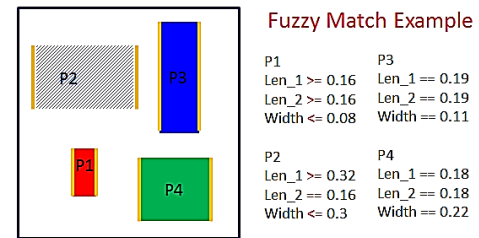


Fig. 10: Any P1 meeting the constraints can be a member of the pattern. Any P2 meeting the constraints can be a member of the pattern.

complexity is $O(k)$ where k is the total number of grid elements overlapped by a search box, Algorithm 2 has $O(nk)$. Because $k \ll n$ or $k = 1$ in general in our pattern match process, it is $O(n)$ in practice. Table II compares these two algorithms and shows substantial run time difference.

E. Fuzzy Pattern Match ²

With our simple approach to pattern match, we could see another benefit of EDDR_PM when it comes to fuzzy pattern matching. Fig. 10 illustrates this. Because we can use not only $==$ but also other relational operators ($>$, $\geq$, $<$, $\leq$) for pattern description, we can describe a pattern in a fuzzy way and do a fuzzy pattern match. In this case, the vector space information is no longer valid. We can use the number of each member inside bounding box for fuzzy pattern match. Therefore, we skip validation checks at the step (24) and (11) of Algorithm 1 for members that are derived from relational operators except $==$ operator. It also must either have at least one member rectangle created by only $==$ operator inside the bounding box or have a configuration where origin member rectangle's center point is unchanging.

²There is a limitation in our approach for fuzzy match. Our approach cannot handle matching from post-OPC (Optical Proximity Correction) pattern to pre-OPC pattern. However, tolerance-based match [14] can be easily accomplished in our approach. Refer to Appendix B.

Algorithm 3 EDDR PM Fuzzy Match

```

1: procedure EDDR-PM(P1, P2, ..., Pn, nonMem, PDB)
2:   Inputs (P1..n, nonMem, and PDB) are the same as Algorithm 1.
3:   ADD_BIN for each member from P1 to Pn.
4:   LOCATE_BIN for all the origin members of P1 and get bin_counts
5:   for  $i = 1 \rightarrow \text{bin\_counts for } p_i \text{ in P1 bins}$  do
6:     LOCATE_BIN a reference member, p2, in P2 bins by searching
7:     the vector distance between P1's center and P2's center.(PDB has this info.)
8:     if found then
9:       if the found p2(reference member) is fuzzy member then
10:        Create one of 8 bounding boxes using 8 different d1, d2, d3,
11:        and d4 sets stored in PDB.
12:        Iterate 8 times from step(19) to step(35).
13:       else
14:         if the found p2 a valid reference member then
15:          Create a bounding box using d1, d2, d3, and d4 determined by
16:          vector info between P1 and the found P2's center.
17:         else
18:          No match. continue to next p1 in P1
19:         if nonMem exists inside the bounding box then
20:          No match. continue to next p1 in P1
21:         LOCATE_BIN for other members in  $P3 \dots P_n$ 
22:         inside the bounding box.
23:         for each member inside the bounding box do
24:            $n = \text{number of each member inside bounding box}$ 
25:            $m = \text{number of each member described in PDB}$ 
26:           if  $n \neq m$  then
27:             No match. continue to next p1 in P1
28:           if member is fuzzy member then
29:             skip member validation check
30:           else
31:             if valid member == false then
32:               No match. continue to next p1 in P1
33:             Matching pattern found at this point.
34:             Output the bounding box to indicate the match.
35:             continue to next p1 in P1
36:         else
37:           No match. continue to next p1 in P1

```

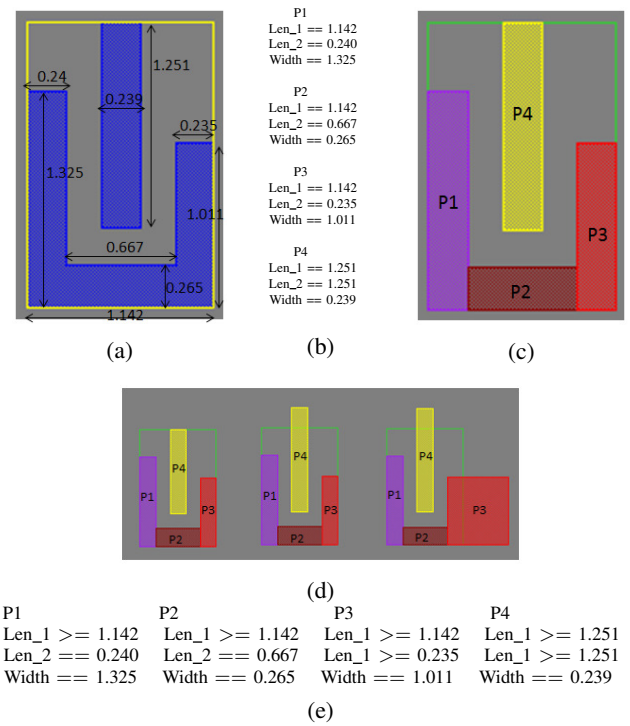


Fig. 11: (a) pattern to match; (b) exact pattern description; (c) member rectangles created from (b) and green bounding box for matched pattern; (d) member rectangles created from (e) and green bounding boxes; (e) fuzzy pattern description.

Since PDB has information about relational operator used for each member, we can decide whether to do fuzzy match or exact match for each member. If == operator is not used for Len_1, Len_2, or Width/Space, we do fuzzy match for that member by skipping member validation check at step (24) of Algorithm 1. If reference member is described as fuzzy member, we skip the step (11) of Algorithm 1 and need to iterate 8 times for fuzzy match. Algorithm 3 is fuzzy match algorithm. Note that step (9) and step (28) in Algorithm 3 for fuzzy member check are added.

Fig. 11 shows fuzzy match examples in details. (b) is for exact match where we have four member rectangles, P1, P2, P3, and P4. (e) uses relational operators to perform fuzzy match. Green bounding boxes are outputs from EDDR_PM when it found matches. Another fuzzy match example is described at Fig. 12 where Space member rectangle is applied.

In the fuzzy match example of Fig. 11, note that P1 and P2's center points are not changed to perform successful fuzzy pattern matching. As long as we have two unchanging center points, one or two iteration is sufficient for pattern match. If there is only one member with its unchanging center point, we have to iterate 8 times to cover all the 8 orientations, which is the case of Fig. 12.

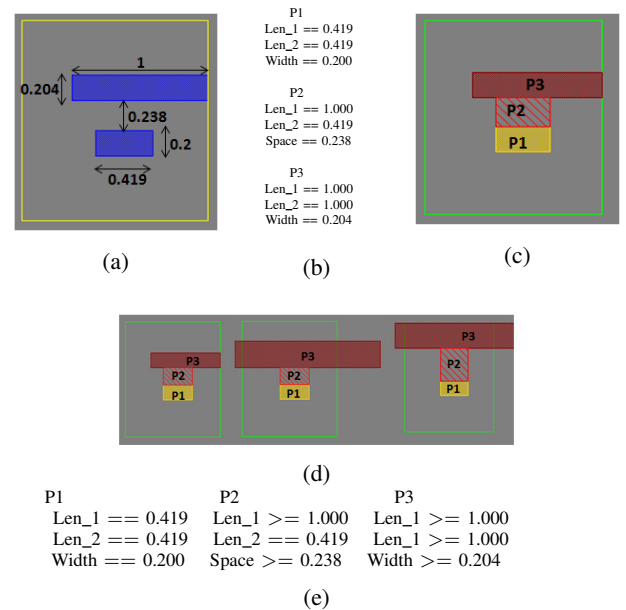


Fig. 12: (a) pattern to match; (b) exact pattern description; (c) member rectangles created from (b) and green bounding box for matched pattern; (d) member rectangles created from (e) and green bounding boxes; (e) fuzzy pattern description.

F. Partial Match

Another interesting case for our approach is when we try to do partial match. For example, we can match a pattern in

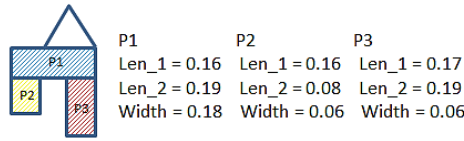


Fig. 13: Partial match example

Fig. 13 by skipping the step (16) in Algorithm 1. Therefore, we can actually match patterns that contain non-orthogonal edges as well. As long as there are a couple of member rectangles in a pattern, we can do partial match, which means it can do match for incompletely-specified patterns. Other approaches for partial match create “Don’t care.” regions. For example, [14] tried to redefine their method to reflect the impacts of “Don’t care” regions for partial match. However, our approach does not need to add additional efforts to deal with a concept of “Don’t care” regions for partial match because it is automatically defined. If some polygons or parts of a polygon are not specified in a bounding box of a pattern like Fig. 13, our algorithm does not care those and perform partial match.

Fig. 14 shows partial match experimental results using “Ind1” test pattern of [1]. Fig. 14 (a) and Fig. 14 (b) illustrates “Ind1” pattern configuration and its pattern description. Fig. 14 (c) is showing partial match results when we skip the step (16) in Algorithm 1. Fig. 14 (d) depicts partial match results when we not only skip it but also we don’t specify P4 in pattern description not to create members for parts of a polygon at the first place as Fig. 14 (e).

IV. EXPERIMENTAL RESULT

Our experiments were performed on a Linux platform with 3.7 GHz clock CPU and 32 GB RAM. We created 9 different patterns to match (Fig. 16). Real industry layout was used for this experiment. The area and number of polygons inside each layout are shown at Table I.

First we compared performance between Algorithm 1 and Algorithm 2. Table II shows the result. This result makes it clear how efficient Algorithm 2 is and at the same time how inefficient the brute force Algorithm 1 is when there are many hotspots to match. It is important to note that the result shown in the table proves our previous assertion: “It scales as well with increasing number of patterns to match because it needs to go through all edges only once in the chip on which it does pattern matching, regardless of the number of patterns to match.”

Note in Table II that # of P1 is equal to number of hotspots in the layout. tp9 4M is a test case where there are 4 million of tp9. DRC1 denotes edge operation time to run through all edges in the layout for finding Len_1 and Len_2 for member rectangles. DRC2 is edge operation time to create member rectangles based on the found edges in DRC1. EDDR column

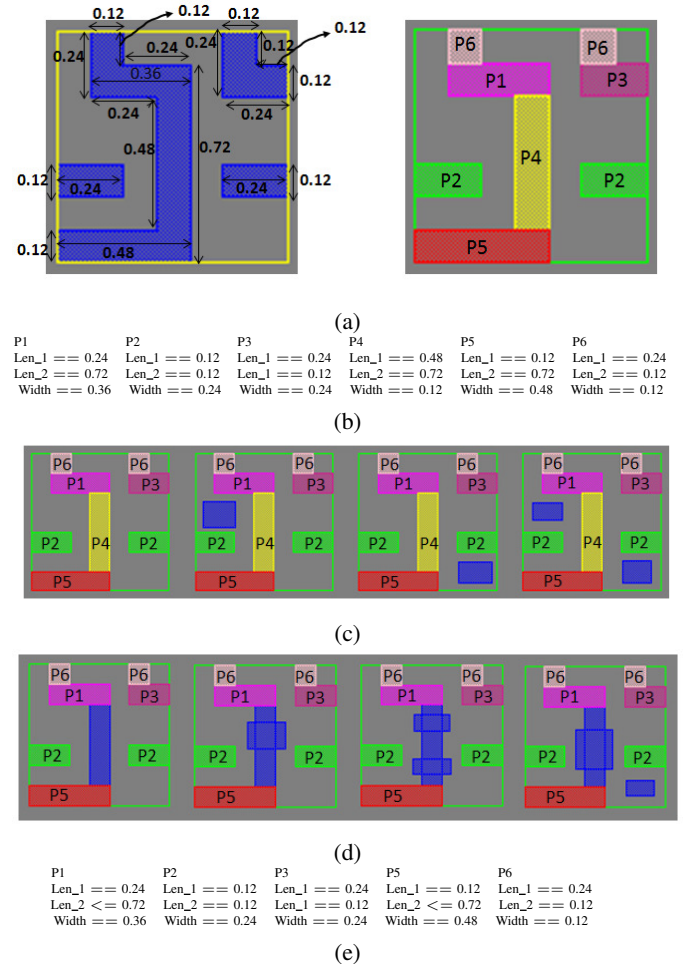


Fig. 14: (a) “Ind1” pattern; (b) pattern description for exact match; (c) partial match results by skipping non-member check; (d) partial match results by both skipping non-member check and removing P4 member creation; (e) pattern description for partial match

means time for Edge Driven Dissected Rectangle Pattern Match. The only difference for EDDR between Algo1 and Algo2 is Algo2 used bin-search grid algorithm while Algo1 is a topological scan-line search algorithm. Success rate is matching success rate. There are no false positive matches. Our approach guarantees 100% accurate match.

Secondly, we compared our Algorithm 2 with a state-of-art DRC-based commercial pattern matching tool which uses the hashing of polygon vertices [10]. We summarized its result in Table III where you can see that our approach is 2 to 10 times faster than the commercial tool.

We also compared [1]’s result to our approach. Because it is not a commercial tool available for us to try, we followed the author’s metrics of [1] to have a fair comparison. As you can see at Table IV, their approach suffers longer run times

when there are too many locations that DRC reports from their critical rules. For example, in their Layout2, they had tp9 called “Stair 1”, and DRC reported less than 4 million of locations of it. But their approach caused a dramatic run time impact. Depending on a pattern configuration, [1]’s approach suffers run time degradation in their Pre-filtering as well as Finalization, while our approach shows fast EDDR_PM time and consistent DRC run time regardless of pattern configurations because we go through all edges only once using simple edge operation rules to create locations of interest.

To be more convinced that our approach is better than [1], we obtained benchmark data from the authors of [1], which included their layout, patterns, and matching results. Fig. 17 presents their test patterns and layout information. For a fair comparison, we set exactly the same experimental settings as their benchmark, using the same 2.4 GHz CPU and 16GB memory on a Linux machine. Table V shows our results against their benchmark data. As we expected, our approach was 20 times faster on average. It was 58 times faster for “Stair1”.

As another data point, there is a recent work [15] that compared exactly same benchmark data of [1] we used in this paper. Table III in [15] shows [15]’s approach is faster than only 5.6 times on average and only about 18 times faster for “Stair1”. Table V includes [15]’s result as well.

V. CONCLUSION

In this paper, we presented a novel methodology for fast and accurate pattern matching. Along with the idea of employing super-fast DRC edge operations to do edge driven dissected rectangles pattern match, we showed how to utilize the mathematical vector concept to avoid the unnecessary 8 iterations for detecting the same pattern in 8 different orientations.

We also presented possible applications of our approach such as fuzzy match and partial match. Our results show that our approach achieves 100% accurate match and it is significantly faster than other methods.

Another point we would like to make is that this technique can be easily adapted in multi-layer pattern matching. For example, if a pattern includes several layers such as metal_1, via, and metal_2, we can use edge operations not only in metal_1 layer but also between those layers to derive member rectangles that match the configuration of those layers and perform EDDR_PM.

APPENDIX A

Definition 1. LENGTH is an edge operation function that takes three inputs (edges of polygons, length constraint, length value) and returns edges that meet the constraint. Length constraint can be any relational operators such as ==, >=,

<=, and etc. It can be expressed:

$E' = \text{LENGTH}(E, RO, \text{value})$

* E' : edges that meet the length constraint.

* E : edges of polygons.

* RO : relational operator.

Definition 2. WIDTH is an edge operation function that takes five inputs (edges of polygons, edges of polygons, all edges of layer, width constraint, width value) and measure width between the first input edges and the second input edges facing inward polygons. It returns rectangles using the edges that meet the width constraint in the form of P1 in Fig. 2 (a). Width constraint can be any relational operators such as ==, >=, <=, and etc. It can be expressed:

$R = \text{WIDTH}(E1, E2, E3, RO, \text{value})$

* R : rectangles formed by edges that meet the width constraint.

* $E1, E2, E3$: edges of polygon. $E1$ and $E2$ are subset of $E3$.

Definition 3. SPACE is the same edge operation function as WIDTH except that it measures space between the first input edges and the second input edges facing outward polygons. It can be expressed:

$R = \text{SPACE}(E1, E2, E3, RO, \text{value})$

Example 1. P1 in Fig. 2 (a) can be created by LENGTH and WIDTH DRC operation.

// a is length for Len_A. Metal_1 is Metal_1 layer’s edges.

// b is length for Len_B. Metal_1 is Metal_1 layer’s edges.

// 0.3 is width value.

Len_A = LENGTH (Metal_1, ==, a)

Len_B = LENGTH (Metal_1, ==, b)

P1 = WIDTH (Len_A, Len_B, Metal_1, ==, 0.3)

Definition 4. ANGLE is an edge operation function that takes three inputs (edges of polygons, angle constraint, angle value) and returns edges that meet the constraint. Angle constraint can be any relational operators such as ==, >=, <=, and etc. It can be expressed:

$E' = \text{ANGLE}(E, RO, \text{value})$

Example 2. Fig. 15 is one of cases that may have different ways to decompose into member rectangles. If we apply “ANGLE == 0” on Len_1, we get Fig. 15 (b). If “ANGLE == 90” on Len_1 is applied, we get Fig 15 (c). DRC operations below are to create members as Fig. 15 (b).

P1_Len_1 = ANGLE (LENGTH (Metal_1, ==, 2), ==, 0)

P1_Len_2 = LENGTH (Metal_1, ==, 2)

P1 = WIDTH (P1_Len_1, P1_Len_2, Metal_1, ==, 2)

P2_Len_1 = ANGLE (LENGTH (Metal_1, ==, 2), ==, 0)

P2_Len_2 = LENGTH (Metal_1, ==, 1)

P2 = WIDTH (P2_Len_1, P2_Len_2, Metal_1, ==, 0.5)

Definition 5. OR is an polygon operation function that arbitrary number of polygons as inputs ($P1, \dots, Pn$) and returns the union of polygon regions. It can be expressed:

$P' = \text{OR}(P1, \dots, Pn)$

* P' : merged polygons.

* $P1, \dots, Pn$: polygons.

Definition 6. NOT is an polygon operation function that

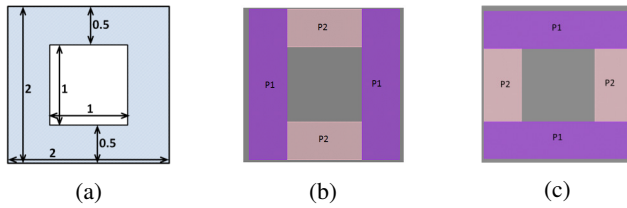


Fig. 15: Example of applying ANGLE (a) pattern that can be decomposed in different ways; (b) ANGLE == 0 on Len_1; (c) ANGLE == 90 on Len_1

takes two inputs (P1,P2) and returns P1 without overlapped region with P2.It can be expressed:

$P' = \text{NOT}(P1, P2)$

* P': P1 polygons without overlapped area with P2.

* P1,P2: polygons.

APPENDIX B

Tolerance-based match

Widely used resolution enhancement technique in lithography process during chip manufacturing may create process-hotspots from patterns which are quite similar as a hotspot pattern and only have tiny width or space differences. Basically, process-hotspot can have slightly different topologies from hotspot patterns for match. Enumerating all these variant topologies as hotspot patterns is not practical, and there must be a representative pattern with edge tolerance and incomplete specified region [14].

Our approach can be easily extended to solve this issue by adding range relational operations along with partial match we presented at Section III-F. For example, a member can be defined as:

$\text{Len}_1 \Rightarrow a \leq b$ (Len_1 is between length a and b.)

$\text{Len}_2 \geq c \leq d$ (Len_2 is between length c and d.)

$\text{Width} \geq e \leq f$ (Width is between width e and f.)

With this range specification, we can specify edge tolerance. By using this edge tolerance and our “Don’t care region” (incomplete specification) approach, we can find process-hotspots as [14].

ACKNOWLEDGMENT

We thank the authors of [1] for providing us with their benchmark data for a fair comparison.

REFERENCES

- [1] Y.-T. Yu, “Accurate process-hotspot detection using critical design rule extraction.” DAC, 2012, pp. 1167–1172.

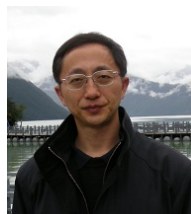
- [2] L. Wang, “Pattern matching for identifying and resolving non-decomposition friendly designs for double patterning technology,” vol. 8684. SPIE, 2013.
- [3] A. Kahng, “Fast dual graph based hotspot detection,” vol. 6349. SPIE, 2006, pp. 628–635.
- [4] D. Ding, “Machine learning based lithographic hotspot detection with critical feature extraction and classification.” ICICDT, 2009, pp. 219–222.
- [5] —, “High performance lithographic hotspot detection using hierarchically refined machine learning.” ASP-DAC, 2011, pp. 775–780.
- [6] J. Wu, “Rapid layout pattern classification.” ASP-DAC, 2011, pp. 781–786.
- [7] H. Yao, “Efficient process-hotspot detection using range pattern matching.” ICCAD, 2006, pp. 625–632.
- [8] J. Xu, “Accurate detection for process-hotspots with vias and incomplete specification.” ICCAD, 2007, pp. 839–846.
- [9] F. Pikus, “Topological pattern matching.” US Patent 018594 A1, July 2010.
- [10] F. Gennari, “Fast pattern matching.” US patent 7818707, Oct. 2010.
- [11] J. Guo, “Improved tangent space based distance metric for accurate lithographic hotspot classification.” ACM/IEEE DAC, 2012, pp. 1173–1178.
- [12] S. Lin, “A novel fuzzy matching model for lithography hotspot detection.” ACM/IEEE DAC, 2012, pp. 1–6.
- [13] H. Yao, “Efficient range pattern matching algorithm for process-hotspot detection.” IET Circuits, Devices and Systems, 2008, pp. 2–15.
- [14] Y.-T. Yu, “Drc-based hotspot detection considering edge tolerance and incomplete specification.” ICCAD, 2014, pp. 101–107.
- [15] H.-Y. Su, “A novel fast layout encoding method for exact multilayer pattern matching with prufer encoding,” *Computer-Aided Design of Integrated Circuits and Systems, IEEE Transactions*, vol. 34, pp. 95–108, 2015.



Jea Woo Park received BS of Physics from Pusan National University in 1998. He worked as a lithography process engineer at Hynix Memory Research and Development until 2003. Since then, he has been working at Mentor Graphics for developing RET and OPC products. Currently working for DFM products as a senior developer. He received his MSCS at Portland State University in 2009. He is a PhD candidate of ECE at Portland State University.



Robert Todd graduated with BSEE/MSEE from MIT in 1980. He developed and supported logic synthesis at IBM from 1980 to 1988. From 1988 to present, he has been a core developer for IC verification products at Mentor Graphics. Currently Chief Scientist in the Design To Silicon division.



Xiaoyu Song received the PhD degree from the University of Pisa, Italy, 1991. From 1992 to 1998, he was on the faculty at the University of Montreal, Canada. He joined the Department of Electrical and Computer Engineering at Portland State University in 1998, where he is now a professor. He was an editor of IEEE Transactions on VLSI Systems and IEEE Transactions on Circuits and Systems. He was awarded an Intel Faculty Fellowship from 2000 to 2005. His research interests include formal methods, design automation, embedded system design and

optimization.

TABLE I: Layout information for TableII and TableIII (*tp9 4M: 4 million of tp9 exist in the layout.)

	Layout for tp1 to tp9	Layout for tp9 4M
Area(mm ²)	1.5 x 1.5	2 x 2
Number of Polygons	5,207,283	9,170,937

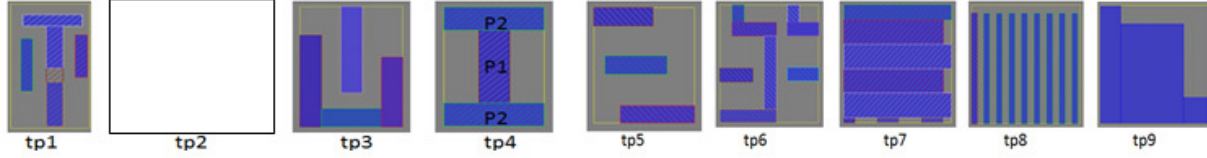


Fig. 16: Test patterns to match for our experiments. tp2 is a clip from the real design of layout 1, which is whited out due to proprietary concerns. tp3, tp4, tp5, tp6, tp9 are from [1].

TABLE II: Algorithm 1 VS. Algorithm 2

	Pattern	# of locations of DRC1	# of locations of DRC2	# of P1	DRC1 (sec)	DRC2 (sec)	EDDR(sec)	Total(sec)	Success rate
Algo 1	tp1	3,298,447	38,663	6,336	40	2	17	59	100%
	tp2	13,772,975	4,627,518	7,219	44	17	25	86	100%
	tp3	1,934,668	115,371	37,581	37	1	172	210	100%
	tp4	488,596	172,476	112,856	37	1	389	427	100%
	tp5	243,343	28,656	9,336	37	1	13	51	100%
	tp6	721,911	154,422	25,518	37	1	111	149	100%
	tp7	337,058	357,696	19,480	39	1	66	106	100%
	tp8	271,188	406,782	15,236	40	1	81	122	100%
	tp9	6,664,603	163,984	40,996	38	1	112	151	100%
	tp9 4M	26,491,623	16,025,600	4,016,400	57	23	205,772	205,852	100%
Algo 2	tp1	3,298,447	38,663	6,336	40	2	5	47	100%
	tp2	13,772,975	4,627,518	7,219	44	17	5	66	100%
	tp3	1,934,668	115,371	37,581	37	1	4	42	100%
	tp4	488,596	172,476	112,856	37	1	4	42	100%
	tp5	243,343	28,656	9,336	37	1	4	42	100%
	tp6	721,911	154,422	25,518	37	1	4	42	100%
	tp7	337,058	357,696	19,480	39	1	4	44	100%
	tp8	271,188	406,782	15,236	40	1	4	45	100%
	tp9	6,664,603	163,984	40,996	38	1	4	43	100%
	tp9 4M	26,491,623	16,025,600	4,016,400	57	23	16	96	100%

TABLE III: Alogrithm2 VS. Commercial Vertices Hashing

Pattern	Algo2 (sec)	Vertices hashing	Speed up
tp1	47	186	4.0
tp2	66	365	5.5
tp3	42	268	6.4
tp4	42	211	5.0
tp5	42	403	9.6
tp6	42	447	10.6
tp7	44	380	8.6
tp8	45	387	8.6
tp9	43	96	2.2
tp9 4M	96	167	1.7

TABLE IV: Alogrithm2 VS. [1]'s critical design rule extraction method

	Pattern	# of location of DRC		# of location after pre-filtering	number of hotspots	DRC(sec)		pre-filtering (sec)	Finalization(sec)	Total(sec)	Success rate
[1]	tp3(mountain)	97,418		9,600	9,600	11		2.98	3.54	17.52	100%
	tp4(I)	67,640		38,400	9,600	9		2.27	56.25	67.52	100%
	tp5(Stair2)	115,950		19,200	9,600	9		3.99	14.18	27.17	100%
	tp6(Ind1)	895,377		9,600	9,600	35		50.44	3.61	89.05	100%
	tp9(Stair1)	3,710,439		57,592	9,600	63		164	125	352	100%
Algo2		DRC1	DRC2			DRC1	DRC2		EDDR(sec)		
	tp3(mountain)	1,934,668	115,371	N/A	37,581	37	1	N/A	4	42	100%
	tp4(I)	488,596	172,476	N/A	112,856	37	1	N/A	4	42	100%
	tp5(Stair2)	243,343	28,656	N/A	9,336	37	1	N/A	4	42	100%
	tp6(Ind1)	721,911	154,422	N/A	25,518	37	1	N/A	4	42	100%
	tp9(Stair1)	6,664,603	163,984	N/A	40,996	38	1	N/A	4	43	100%
	tp9(Stair1) 4M	26,491,623	16,025,600	N/A	4,016,400	57	23	N/A	16	96	100%

	Layout
Area (mm ²)	1.0×1.0
#Polygon*	661,056

*#Polygon: the number of polygons.
The layouts are based on 32nm process.



Fig. 17: [1]'s layout information and test patterns

TABLE V: Alogrithm2 VS. [1]'s critical design rule extraction method

[1]						[15]			Algorithm 2				
Pattern	# of hotspots	DRC (sec)	pre-filtering + finalization(sec)	Total (sec)	Success Rate	Total (sec)	Success Rate	Speedup	DRC1 + DRC2	EDDR	Total (sec)	Success Rate	Speedup
Mountain	12,800	14.83	7.09	21.92	100%	19.9	100%	1.1	4	0.2	4.2	100%	5.2
S	12,800	45.89	86.00	131.89	100%	22.5	100%	5.9	8	0.5	8.5	100%	15.5
Stair1	12,800	45.81	318.93	364.74	100%	19.9	100%	18.3	6	0.3	6.3	100%	57.9
I	12,800	10.74	52.47	63.21	100%	20.1	100%	3.1	3	0.2	3.2	100%	19.8
Ind1	12,800	46.68	69.76	116.44	100%	21.6	100%	5.4	5	0.5	5.5	100%	21.2
Ind2	12,800	55.83	43.63	99.46	100%	23.5	100%	4.2	5	0.5	5.5	100%	18.1
Stair2	12,800	9.61	17.57	27.18	100%	19.7	100%	1.4	4	0.2	4.2	100%	6.5